

TradePro Business Logic Specification

Complete Extraction of Trading System Logic

This document provides a comprehensive analysis of the TradePro paper trading platform's business logic, covering trade execution rules across Equity, Futures, and Options segments, balance validation mechanisms, target and stop-loss trigger detection (both server-side and client-side), P&L calculation formulas, the complete order lifecycle from placement to closure, and critical edge cases including race conditions, margin tracking inconsistencies, and auto-creation of option instruments.

VERSION 1.0
DATE June 1, 2026
PLATFORM TradePro Indian Market
CLASSIFICATION Internal Technical Reference

Table of Contents

1. System Overview	2
2. Trade Execution Rules	3
3. Balance Validation	5
4. Target / Stop Loss Logic	6
5. P&L Calculation	8
6. Order Lifecycle	9
7. Edge Cases & Hidden Conditions	11

1. System Overview

TradePro is a paper trading platform designed to simulate the Indian stock market experience without involving real money or requiring SEBI registration. The platform supports three primary market segments: Equity (cash market), Futures (derivative contracts), and Options (CE/PE contracts on indices and F&O stocks). All trading is conducted using virtual currency, with each user receiving a default allocation of Rs.1,00,000 upon registration. The system enforces realistic market rules including brokerage calculations, margin requirements, market hours, and circuit limits, while providing a safe learning environment for users to practice trading strategies.

The business logic is split across two execution environments: the **server-side** (Next.js API routes) handles order placement, position management, balance validation, and transactional integrity via Prisma ORM with SQLite/PostgreSQL; the **client-side** (ExecutionEngine running in the browser) handles real-time stop-loss and target monitoring against simulated market ticks generated by the MarketEngine. This split exists because live simulated prices only exist in the client-side MarketEngine, so server-side SL/TP checks would never see the latest price. The ExecutionEngine bridges this gap by detecting trigger conditions locally and calling server APIs to execute square-offs.

1.1 Core System Parameters

Parameter	Value	Source	Configurable
Default Virtual Balance	Rs.1,00,000	User model default	No (hardcoded)
Brokerage Rate	0.05% of total value	BROKERAGE_PERCENT env	Yes
Minimum Brokerage	Rs.20	MIN_BROKERAGE env	Yes
Maximum Brokerage	Rs.500	MAX_BROKERAGE env	Yes
Default Futures Margin	12%	DEFAULT_FUTURES_MARGIN_PERCENT	Yes
Default Options Short Margin	150%	DEFAULT_OPTIONS_SHORT_MARGIN_PERCENT	Yes
Max Order Volume	10,000 qty	MAX_ORDER_VOLUME env	Yes
Market Hours (IST)	09:15 - 15:30	checkMarketStatus()	No (hardcoded)
SL/TP Check Rate Limit	20 seconds/user	check-triggers route	No (hardcoded)
Floating Point Tolerance	Rs.0.01 (1 paisa)	ExecutionEngine	No (hardcoded)

Table 1: Core System Parameters

2. Trade Execution Rules

Trade execution in TradePro follows a strict pipeline: authentication, input validation, market status check, price resolution, balance/margin verification, and transactional execution. The system differentiates between MARKET orders (executed immediately at the current market price) and pending order types (LIMIT, SL, SL_M) which are stored with PENDING status and executed later when trigger conditions are met. All successful executions result in the creation of an Order record (status FILLED), a Trade record, and either the creation of or update to a Position record, all within a single Prisma database transaction to ensure atomicity.

2.1 Order Types and Trigger Conditions

The platform supports four order types, each with distinct execution semantics. MARKET orders execute immediately at the prevailing market price. LIMIT orders wait until the market price reaches the specified limit price or better. SL (Stop Loss) orders activate when the trigger price is reached, then execute as a limit order at the specified price. SL_M (Stop Loss Market) orders activate at the trigger price and execute at market price. For LIMIT orders on the BUY side, the trigger condition is $currentPrice \leq order.price$; for SELL side, $currentPrice \geq order.price$. For SL and SL_M orders on the BUY side, the trigger condition is $currentPrice \geq triggerPrice$; for SELL side, $currentPrice \leq triggerPrice$.

Order Type	BUY Trigger	SELL Trigger	Execution Price
MARKET	Immediate	Immediate	Current market price
LIMIT	$price \leq limitPrice$	$price \geq limitPrice$	Current market price at trigger
SL	$price \geq triggerPrice$	$price \leq triggerPrice$	Current market price at trigger
SL_M	$price \geq triggerPrice$	$price \leq triggerPrice$	Current market price at trigger

Table 2: Order Types and Trigger Conditions

2.2 Segment-Specific Execution Logic

2.2.1 Equity Segment

In the Equity segment, BUY orders require the full order value plus brokerage to be available in the user's virtual balance. The required amount is calculated as: **requiredAmount = totalValue + brokerage**, where $totalValue = quantity * fillPrice$. Upon execution, the system deducts the required amount from virtualBalance, increments totalTrades, and decrements totalPnl by the brokerage amount (since brokerage is an immediate realized cost). The system then checks for an existing open position with matching userId, symbol, segment, productType, and tradeDirection. If found, the position is averaged up: the new entry price is calculated as the weighted average of the existing investment and the new trade value. If no existing position exists, a new Position record is created with entryPrice equal to fillPrice, totalInvested equal to totalValue, and unrealizedPnl calculated as $(currentValue - totalInvested)$.

For Equity SELL orders (MARKET type), the system enforces that the user must hold an open BUY position for the same symbol, segment, and productType. It checks that $\text{position.quantity} \geq \text{order.quantity}$; otherwise the order is rejected with "Insufficient quantity". The realized P&L on sell is calculated as: **$\text{realizedPnl} = (\text{fillPrice} - \text{entryPrice}) * \text{quantity} - \text{brokerage}$** . The sell proceeds ($\text{totalValue} - \text{brokerage}$) are credited to virtualBalance. If the sell quantity equals the position quantity, the position is closed ($\text{isOpen} = \text{false}$, $\text{exitReason} = \text{MANUAL}$). If partial, the position is updated with reduced quantity and adjusted totalInvested.

2.2.2 Futures Segment

Futures trading operates on a margin-based system. Instead of deducting the full contract value, only the margin requirement is blocked from the user's virtual balance. The margin is calculated as: **$\text{marginRequired} = \text{totalValue} * \text{marginPercent} / 100$** , where marginPercent varies by underlying instrument (e.g., NIFTY: 12%, BANKNIFTY: 14%, individual stocks: 14-25%). The lotSize is resolved from the Index table (for index futures) or the Future contract itself, defaulting to 50 if not found. Total quantity is calculated as: **$\text{totalQty} = \text{lots} * \text{lotSize}$** . On execution, the system deducts marginRequired from virtualBalance and increments marginUsed by the same amount. Notably, brokerage is calculated on the full contract value ($\text{totalQty} * \text{fillPrice}$) but is NOT deducted from the balance at entry time for futures. The margin is only released when the position is squared off. For SELL futures (short positions), the same margin logic applies; the system requires margin rather than an existing position, enabling both long and short futures trading.

Underlying	Lot Size	Margin %
NIFTY	50	12%
BANKNIFTY	25	14%
FINNIFTY	25	13%
SENSEX	15	12%
MIDCPNIFTY	75	15%
RELIANCE	250	16%
TCS	150	15%
HDFCBANK	550	14%
ADANI	500	25%
TATASTEEL	4250	22%

Table 3: Futures Margin and Lot Size Configuration

2.2.3 Options Segment

Options trading supports both BUY (long) and SELL (short/write) positions for CE (Call European) and PE (Put European) option types. BUY options require the full premium amount plus brokerage:

requiredAmount = **totalValue** + **brokerage**, where $\text{totalValue} = \text{lots} * \text{lotSize} * \text{fillPrice}$. SELL (short) options require margin, calculated similarly to futures: **marginRequired** = **totalValue** * **marginPercent** / **100**. The default short option margin is 150% of premium value. A critical feature of the Options segment is the **auto-creation of option instruments**. When a user places an options trade for a strike/expiry combination that does not yet exist in the database, the system automatically creates the Option record on-the-fly using the price provided in the request body. The auto-created option receives default Greeks values (delta: +0.5 for CE, -0.5 for PE; gamma: 0.01; theta: -2.5; vega: 5.0), a random implied volatility between 15-30%, and an expiry date set to the nearest Thursday (Indian weekly expiry convention). The lotSize for options is resolved from the Index table, defaulting to 50 if not found. Position matching for options includes optionType and strikePrice in the unique key, allowing the same user to hold separate positions for the same underlying at different strikes or option types simultaneously.

3. Balance Validation

Balance validation is a critical gate in the trade execution pipeline. Every order must pass balance verification before execution, and the system re-checks balance when pending orders are triggered by the check-triggers endpoint. The validation logic varies by segment and trade direction, reflecting the different capital requirements for cash purchases versus margin-based derivatives trading.

3.1 Validation Rules by Segment and Direction

Segment	Direction	Required Amount	Check
EQUITY	BUY	totalValue + brokerage	virtualBalance >= required
EQUITY	SELL	N/A (position required)	position.quantity >= order.quantity
FUTURES	BUY	marginRequired	virtualBalance >= marginRequired
FUTURES	SELL	marginRequired	virtualBalance >= marginRequired
OPTIONS	BUY	totalValue + brokerage	virtualBalance >= required
OPTIONS	SELL	marginRequired	virtualBalance >= marginRequired

Table 4: Balance Validation Rules by Segment and Direction

3.2 Balance Impact on Trade Execution

When a MARKET order passes validation, the balance impact is applied immediately within the same database transaction. For Equity BUY: virtualBalance is decremented by (totalValue + brokerage), totalTrades is incremented, and totalPnl is decremented by brokerage. For Equity SELL: virtualBalance is incremented by (totalValue - brokerage) which represents the net proceeds after brokerage deduction, totalTrades is incremented, and totalPnl is incremented by the realized P&L. For Futures BUY/SELL: virtualBalance is decremented by marginRequired, marginUsed is incremented by marginRequired, and

totalTrades is incremented. Brokerage is NOT deducted from virtualBalance at entry for futures; it is accounted for when the position is squared off. For Options BUY: same as Equity BUY pattern. For Options SELL (short): same as Futures margin pattern.

3.3 Pending Order Balance Re-Validation

When a pending order (LIMIT/SL/SL_M) is triggered by the check-triggers endpoint, the system re-validates the user's balance at trigger time. This is crucial because the user's financial situation may have changed between order placement and trigger. If the balance check fails at trigger time, the order is marked as REJECTED with an appropriate rejectReason (e.g., "Insufficient balance at trigger time" or "No open position at trigger time"). The check-triggers endpoint fetches the user record once at the beginning and uses it for all order validations within that batch, avoiding per-order DB queries for performance. However, this means that if multiple triggered orders would each individually pass but collectively exceed the balance, only the first ones to be processed will succeed. This is a known edge case where order processing order matters and later orders may be rejected even though the user had sufficient balance when the trigger check began.

4. Target / Stop Loss Logic

The Target (Take Profit) and Stop Loss system operates through two distinct mechanisms: a **server-side check-triggers endpoint** that runs periodically, and a **client-side ExecutionEngine** that monitors real-time simulated prices. The dual mechanism exists because the server-side database does not contain live simulated prices; those only exist in the client-side MarketEngine. Therefore, the ExecutionEngine runs in the browser, subscribes to live market ticks, and detects SL/TP triggers against real-time prices. When a trigger fires, it calls the server API to execute the square-off.

4.1 SL/TP Placement Validation

When placing an order with stopLoss and/or target values, the system enforces strict directional validation to prevent invalid configurations where the SL or TP would trigger immediately. The validation rules are as follows:

Direction	Stop Loss Rule	Target Rule	SL vs Target
BUY	SL < entryPrice	Target > entryPrice	SL < Target
SELL	SL > entryPrice	Target < entryPrice	SL > Target

Table 5: SL/TP Directional Validation Rules

Additionally, the system enforces a minimum distance of Rs.0.01 (1 paise) between the entry price and both the stop loss and target values. This prevents orders where the SL or TP is placed at the exact entry price, which would trigger immediately upon execution. The entry price used for SL/TP validation during order placement is the "price" parameter from the request body (or the current market price for MARKET orders). For LIMIT/SL/SL_M orders, this is the specified limit/trigger price, which means the validation uses the

intended entry price rather than the actual fill price that will only be known at execution time. This creates a subtle edge case: if a LIMIT BUY order specifies a price significantly below the current market price, the SL/TP validation uses that limit price as the entry reference, even though the actual fill price may differ when the order triggers.

4.2 Server-Side SL/TP Trigger Detection (check-triggers)

The `/api/trade/check-triggers` endpoint performs two types of checks: (1) PENDING LIMIT/SL order trigger detection, and (2) open position SL/TP monitoring. The endpoint is rate-limited to one call per user per 20 seconds to prevent excessive database queries. For position SL/TP monitoring, the endpoint fetches all open positions for the user that have a non-null, positive stopLoss or target value (limited to 50 positions per check). It then batch-fetches current prices from the database for all relevant stocks, futures, and options in parallel using `Promise.all`. The trigger logic for positions is straightforward: for BUY positions, if `currentPrice <= stopLoss`, the stop loss triggers; if `currentPrice >= target`, the target triggers. For SELL positions, if `currentPrice >= stopLoss`, the stop loss triggers; if `currentPrice <= target`, the target triggers. Stop loss is checked before target; if SL triggers, target is not evaluated for that position in the same check cycle.

4.3 Client-Side ExecutionEngine (Real-Time Monitoring)

The ExecutionEngine is a client-side singleton that runs in the browser and monitors all open positions against live market ticks generated by the MarketEngine. It employs two complementary detection methods to catch triggers even during rapid price movements:

Method 1 - Cross-Price Check: Compares the previous checked price against the current price to detect if a trigger level was crossed between ticks. For BUY target: triggers if `prevPrice < target AND currentPrice >= target - 0.01`. For BUY SL: triggers if `prevPrice > stopLoss AND currentPrice <= stopLoss + 0.01`. For SELL target: triggers if `prevPrice > target AND currentPrice <= target + 0.01`. For SELL SL: triggers if `prevPrice < stopLoss AND currentPrice >= stopLoss - 0.01`. The 0.01 threshold is a floating-point tolerance to handle 1-paisa precision.

Method 2 - OHLC Bar Check: Maintains an OHLC (Open-High-Low-Close) bar per symbol that resets every 5 ticks. Even if the cross-price check misses a trigger (because the price quickly moved past and back within a single tick interval), the OHLC bar captures the intra-period high and low. For BUY: target is hit if `bar.high >= target`, SL is hit if `bar.low <= stopLoss`. For SELL: target is hit if `bar.low <= target`, SL is hit if `bar.high >= stopLoss`.

When a trigger is detected, the ExecutionEngine calculates the P&L and emits a `TriggerResult` event to all subscribed listeners. It also adds the position ID to a `squaredOffPositions` set to prevent duplicate triggers for the same position. The UI layer then calls the `/api/trade/square-off` endpoint to execute the actual position closure on the server. This design means that if the user closes the browser tab, the client-side ExecutionEngine stops running and SL/TP monitoring ceases until the user returns and the check-triggers endpoint resumes server-side monitoring.

5. P&L Calculation

Profit and Loss calculation in TradePro follows a consistent formula across all segments, with variations based on trade direction and the specific closing mechanism (manual square-off, SL/TP trigger, or pending order execution). All P&L values are rounded to 2 decimal places using $\text{Math.round}(\text{value} * 100) / 100$ at every calculation step to maintain precision consistency.

5.1 Realized P&L Formulas

Scenario	Formula	Notes
BUY position closed	$(\text{exitPrice} - \text{entryPrice}) * \text{quantity} - \text{brokerage}$	Profit when exit > entry
SELL position closed	$(\text{entryPrice} - \text{exitPrice}) * \text{quantity} - \text{brokerage}$	Profit when entry > exit
Equity SELL (place route)	$(\text{fillPrice} - \text{position.entryPrice}) * \text{quantity} - \text{brokerage}$	Uses original entry price
P&L % (Trade record)	$\text{realizedPnl} / (\text{entryPrice} * \text{quantity}) * 100$	Based on entry cost
P&L % (Position close)	$\text{realizedPnl} / \text{totalInvested} * 100$	Based on total invested

Table 6: Realized P&L Calculation Formulas

5.2 Unrealized P&L Calculation

Unrealized P&L is calculated in two contexts: (1) when a position is created or updated during trade execution, and (2) when the portfolio endpoint enriches positions with live prices. The formula is consistent: for BUY positions, $\text{unrealizedPnl} = (\text{currentPrice} - \text{entryPrice}) * \text{quantity}$; for SELL positions, $\text{unrealizedPnl} = (\text{entryPrice} - \text{currentPrice}) * \text{quantity}$. The $\text{unrealizedPnlPercent}$ is calculated as $\text{unrealizedPnl} / \text{totalInvested} * 100$. When positions are averaged up (additional buys into an existing position), the entryPrice becomes the weighted average, and totalInvested is the sum of all investments. The unrealized P&L is NOT reduced by brokerage at the unrealized stage; brokerage is only applied when the position is actually closed and the P&L is realized.

5.3 Brokerage Calculation

Brokerage is calculated using the formula: **brokerage = max(minBrokerage, min(maxBrokerage, totalValue * brokeragePercent))**, where totalValue is the gross trade value (quantity * fillPrice). The default brokerage rate is 0.05% (BROKERAGE_PERCENT = 0.0005), with a minimum of Rs.20 (MIN_BROKERAGE) and a maximum of Rs.500 (MAX_BROKERAGE). This means that for trades below Rs.40,000 (where 0.05% would be less than Rs.20), the minimum brokerage of Rs.20 applies. For trades above Rs.1,00,00,000 (where 0.05% would exceed Rs.500), the maximum brokerage of Rs.500 applies. Brokerage is applied on BOTH sides of a trade: when entering a position (for Equity BUY and Options BUY, deducted from virtualBalance) and when exiting (deducted from the sell proceeds or margin return). For Futures and Options SELL (short), brokerage is calculated and stored in the order/trade records but is NOT deducted from virtualBalance at entry; it is accounted for when the position is squared off

through the margin return calculation.

5.4 Portfolio-Level P&L

The portfolio endpoint aggregates P&L across all open positions and historical trades. The total portfolio value is calculated as: **totalPortfolioValue** = **virtualBalance** + **totalCurrentValue**, where **totalCurrentValue** is the sum of (quantity * currentPrice) for all open positions across segments. The total return is calculated as: **totalReturn** = (**totalPortfolioValue** - **initialCapital**) / **initialCapital** * **100**, where **initialCapital** is hardcoded as Rs.1,00,000. Total realized P&L is obtained from an aggregate query on closed positions: **totalRealizedPnl** = **SUM(realizedPnl) WHERE isOpen = false**. The available margin is calculated as: **availableMargin** = **max(0, virtualBalance - marginUsed)**. Segment-level breakdowns (equity, futures, options) are computed by grouping positions by their segment field and summing invested, currentValue, unrealizedPnl, and marginUsed within each group.

6. Order Lifecycle

Every trade in TradePro goes through a well-defined lifecycle from placement to completion. The Order model tracks this lifecycle through its status field, which can take one of six values: PENDING, PARTIALLY_FILLED, FILLED, CANCELLED, REJECTED, or EXPIRED. However, in the current implementation, PARTIALLY_FILLED and EXPIRED statuses are defined in the schema but not actively used; all MARKET orders are either FILLED or rejected immediately, and all pending orders transition directly to FILLED or REJECTED upon trigger.

6.1 Lifecycle State Machine

State	Entry Condition	Exit Transitions	Timestamps Set
PENDING	LIMIT/SL/SL_M order placed	FILLED, CANCELLED, REJECTED	placedAt
FILLED	MARKET order executed, or pending order triggered	Terminal state	filledAt
CANCELLED	User cancels pending order	Terminal state	cancelledAt
REJECTED	Balance/position check fails at trigger time	Terminal state	cancelledAt
PARTIALLY_FILLED	Defined in schema, not currently used	N/A	N/A
EXPIRED	Defined in schema, not currently used	N/A	N/A

Table 7: Order Lifecycle States

6.2 Step-by-Step: MARKET Order Flow (Place to Execute)

Step 1 - Authentication: The request token is verified via `authenticateRequest()`. This checks the JWT validity, session existence and expiry, token blacklist status, and user active status. Cached auth results are used when available (`CacheTTL.AUTH`) to avoid repeated database queries.

Step 2 - Input Validation: Required fields (symbol, direction, orderType, segment, productType, quantity) are verified. Direction must be BUY or SELL; orderType must be MARKET, LIMIT, SL, or SL_M; segment must be EQUITY, FUTURES, or OPTIONS; productType must be INTRADAY, DELIVERY, or CARRY_FORWARD. SL/TP directional validation is performed. Quantity must be a positive integer not exceeding `MAX_ORDER_VOLUME` (default 10,000).

Step 3 - Market Status Check: If `ENFORCE_MARKET_HOURS` is set to "strict", the system checks whether the market is currently open (9:15-15:30 IST, weekdays, excluding holidays). If set to "true", a warning is attached to the response but the trade is still allowed (paper trading friendly). If unset or "false", no market hours check is performed.

Step 4 - User Verification: The user record is fetched from the database. If the user does not exist or is not active (`isActive = false`), the order is rejected.

Step 5 - Price Resolution: The fill price is determined based on the segment and order type. For EQUITY MARKET orders, it is the stock's `currentPrice` from the cache or database. For FUTURES, it is the future contract's `ltp`. For OPTIONS, it is the option's `ltp` or the request body price if auto-creating the option. For LIMIT/SL/SL_M orders, the price comes from the request body.

Step 6 - Balance/Margin Check: Depending on the segment and direction, the appropriate balance or margin validation is performed (see Section 3).

Step 7 - Transactional Execution: All database operations are wrapped in a Prisma `$transaction` to ensure atomicity. Within the transaction: (a) user balance is updated, (b) Order record is created with status FILLED, (c) Trade record is created with execution details, (d) Position record is either created or updated. If any step fails, the entire transaction is rolled back.

Step 8 - Cache Invalidation: After successful execution, relevant caches are invalidated: user balance cache (`ubal:userId` prefix) and auth cache (if token was used). This ensures subsequent requests see the updated balance.

6.3 Step-by-Step: Position Close Flow (Square-Off)

Position closure can occur through three mechanisms: (1) manual square-off via `/api/trade/square-off`, (2) SL/TP trigger via `/api/trade/check-triggers` or `ExecutionEngine`, and (3) direct sell via `/api/trade/place` with `direction=SELL` for equity positions. The square-off flow begins by finding the open position that matches the `positionId` and `userId`. The current price is resolved from cache or database based on the position's segment. Realized P&L is calculated using the standard formulas (see Section 5). A closing order (MARKET, opposite direction) and a closing trade record are created within a transaction. The position is marked as closed (`isOpen = false`, `squaredOffAt = now`, `currentValue = 0`, `unrealizedPnl = 0`). Balance is updated differently based on the original trade direction: for BUY positions, `proceeds = totalValue - brokerage` are added to `virtualBalance`; for SELL positions, `marginReturn = marginUsed + realizedPnl` is added to `virtualBalance`. In both cases, `marginUsed` is decremented by the position's `marginUsed` value. The `exitReason` is set based on the trigger: MANUAL for manual square-off, STOP_LOSS or TARGET for

SL/TP triggers.

6.4 Order Modification and Cancellation

Pending orders (status = PENDING) can be modified via /api/trade/modify. The modify endpoint supports updating the price, stopLoss, and target fields for pending orders, as well as stopLoss and target for open positions. Validation is minimal for modifications: price must be > 0, SL and target must be > 0. The directional validation that applies at order placement time is NOT re-applied on modification, which means a user could theoretically modify a pending BUY LIMIT order's stopLoss to a value above the entry price, creating an invalid configuration. This is a known gap in the validation logic. Order cancellation via /api/trade/cancel simply sets the order status to CANCELLED and records cancelledAt, but only for PENDING orders. FILLED, CANCELLED, or REJECTED orders cannot be cancelled. No balance or position changes are required for cancellation since no funds were blocked for LIMIT/SL orders at placement time.

7. Edge Cases & Hidden Conditions

The following section documents non-obvious behaviors, race conditions, and edge cases discovered through code analysis. These are not documented in the user-facing UI and may lead to unexpected behavior if not properly understood.

7.1 Options Auto-Creation: Price Source Trust Issue

When a user trades an option that does not exist in the database, the system auto-creates it using the price from the request body (price || body.ltp || 0). This means the client controls the initial price of the auto-created option. While this is a paper trading platform and the price will quickly be overwritten by the MarketEngine's simulated ticks, there is a brief window where the auto-created option's ltp reflects the user-provided price. If the user provides an artificially low price for a CE option or high price for a PE option, the fill price for a MARKET order would use this artificial ltp, potentially allowing instant profit. The system does not validate the provided price against any market reference at creation time. The auto-created option also receives randomized volume (100,000-600,000) and open interest (20-100 in lakhs), and default Greeks values that may not accurately reflect the actual moneyness of the strike.

7.2 Equity SELL Requires Matching BUY Position (No Naked Short Selling)

In the Equity segment, SELL orders require an existing open BUY position with matching symbol, segment, and productType. The system explicitly checks: position.quantity >= order.quantity. This means naked short selling is not supported for equities. However, for FUTURES and OPTIONS, both BUY and SELL can create new positions without requiring an opposite position, effectively enabling short selling in derivatives. The system also does not check for existing SELL positions when a new BUY order is placed for the same symbol/segment/productType in equity. If a user holds a BUY position and places a new SELL order via the place route (not square-off), the system will find the existing BUY position and reduce it, rather than creating a separate SELL position. This is because the SELL equity logic explicitly searches for an open BUY position to close.

7.3 Position Averaging: SL/TP Override Behavior

When adding to an existing position (averaging up/down), the new order's stopLoss and target values will override the existing position's SL/TP values if provided. The code uses: **stopLoss: stopLoss || existingPosition.stopLoss**. Due to JavaScript's truthy evaluation, if the new order specifies stopLoss = 0, the existing stopLoss is preserved (0 is falsy). However, if the new order specifies any positive stopLoss value, it completely replaces the old one, even if the old value was specifically set for the original entry price. This means a user who adds to a position at a different price point with a new SL will lose their original SL, which may have been calibrated to the original entry price. The entry price itself is recalculated as the weighted average: $\text{newEntry} = (\text{existingTotalInvested} + \text{newTotalValue}) / \text{newTotalQuantity}$, which is correct for P&L calculation but may not align with the new SL/TP values.

7.4 Futures/Options Brokerage Not Deducted at Entry

For Futures and Options SELL (short) positions, the brokerage is calculated and stored in the Order and Trade records but is NOT deducted from the user's virtualBalance at the time of position opening. Only the marginRequired amount is deducted. The brokerage is effectively "absorbed" into the margin return calculation when the position is squared off. For a BUY futures/options position, the pattern differs: margin is deducted but brokerage is also NOT explicitly deducted from virtualBalance. The totalPnl field on the User model is not updated with the brokerage cost at entry for futures (unlike equity BUY where totalPnl is decremented by brokerage). This means the totalPnl metric may be overstated for users with open futures/options positions, as the brokerage cost will only be reflected when the positions are closed.

7.5 Sell Proceeds Calculation: Net of Brokerage

When closing a BUY position via square-off, the proceeds credited to virtualBalance are calculated as: **proceeds = totalValue - brokerage**. The realized P&L is calculated as: **realizedPnl = (exitPrice - entryPrice) * quantity - brokerage**. Then totalPnl on the User model is incremented by realizedPnl. This means the user's virtualBalance receives the gross proceeds minus brokerage, and totalPnl reflects the net P&L after brokerage. The virtualBalance change can be decomposed as: original investment returned + profit - brokerage. For SELL positions closed via square-off, the margin return is: **marginReturn = marginUsed + realizedPnl**. Since realizedPnl already accounts for brokerage, the net effect is correct: the user gets back their margin plus the net profit (or minus the net loss) including brokerage deduction.

7.6 Race Condition: Multiple SL/TP Triggers on Same Position

The ExecutionEngine prevents duplicate triggers through the squaredOffPositions set, which tracks positions that have already triggered. Once a position's SL or TP fires, its ID is added to this set and subsequent price ticks will not re-trigger it. However, there is a potential race condition between the client-side ExecutionEngine and the server-side check-triggers endpoint. Both can independently detect the same trigger condition for the same position. If the ExecutionEngine fires first and calls square-off, the server-side check-triggers will find the position already closed (isOpen = false) and skip it. But if both detect the trigger simultaneously before either completes the square-off, there is a theoretical possibility of the position being squared off twice, resulting in a double credit to the user's balance. The server-side square-off endpoint does check for isOpen = true before proceeding, which provides some protection, but the check and the update are not atomic without a database-level lock.

7.7 Partial Sell: Remaining Position Investment Calculation

When a user sells fewer shares than the total position quantity (partial sell), the remaining position's `totalInvested` is recalculated as: **`newInvested = position.totalInvested - (position.entryPrice * quantity)`**. This uses the average entry price multiplied by the sold quantity, which is correct for FIFO-like accounting. However, the realized P&L is calculated on the entire sold quantity using the same average entry price, regardless of whether the position was built through multiple entries at different prices. Since positions are averaged (not tracked as separate lots), partial sells always use the weighted average entry price for P&L calculation. The remaining position retains its average entry price and the new `totalInvested` reflects the reduced position size at the same average cost.

7.8 Market Hours Enforcement Modes

The `ENFORCE_MARKET_HOURS` environment variable has three behaviors that are not obvious from the UI: When set to "strict", trades outside market hours are blocked with a 403 error. When set to "true", trades outside market hours are allowed but a warning string is attached to the response JSON. When unset or set to any other value, no market hours check is performed at all. The market status check itself involves a database query for market holidays and has a 60-second cache TTL. For Muhurat Trading sessions (special Diwali trading hours), the holiday record has `isMuhurat = true` with `muhuratStart` and `muhuratEnd` times, but the current `checkMarketStatus()` function does not handle Muhurat sessions; it treats Muhurat days as closed because the `isMuhurat` check only excludes non-Muhurat holidays. Muhurat trading hours would need separate handling to allow trading during the special session window.

7.9 Margin Can Go Negative

The `availableMargin` calculation is: **`availableMargin = max(0, virtualBalance - marginUsed)`**. The `max(0, ...)` clamp prevents displaying negative available margin to users. However, the underlying `virtualBalance - marginUsed` can indeed go negative. This can happen when a position accrues unrealized losses that are reflected in `virtualBalance` through a P&L adjustment but the `marginUsed` remains at the original blocked amount. It can also occur if a user's `totalPnl` is decremented (e.g., through brokerage on equity buys) while `marginUsed` stays high from open futures/options positions. The system does not perform a margin call or force-close positions when available margin goes negative; it only prevents new margin-requiring trades by checking `virtualBalance >= marginRequired` at order time.

7.10 Pending Orders Do Not Block Balance

When a `LIMIT`, `SL`, or `SL_M` order is placed, no funds are reserved or blocked from the user's virtual balance. The order simply sits in `PENDING` status with no financial impact until triggered. This means a user can place multiple pending orders that, if all triggered simultaneously, would exceed their available balance. The `check-triggers` endpoint re-validates balance at trigger time, but as noted in Section 3.3, it fetches the user balance once at the start of the batch. If multiple pending orders trigger in the same batch, the first orders to be processed may consume the balance, causing later orders in the same batch to be rejected. The processing order follows the database query order (by ID, effectively creation order), which may not align with the user's intended priority.

7.11 Trigger Price vs Order Price Confusion for SL Orders

When placing SL or SL_M orders, the system stores the "price" parameter as both the order.price and (for SL/SL_M types) as order.triggerPrice. The code is: `triggerPrice: (orderType === "SL" || orderType === "SL_M") ? price : null`. This means the same value serves as both the trigger price and the order price. In a real trading system, SL orders typically have separate trigger and limit prices (the trigger price activates the order, and the limit price is the worst price at which to execute). In TradePro, when the trigger fires, the order executes at the current market price, not at the original "price" value. This simplification means that SL orders behave like SL-M (Stop Loss Market) orders regardless of the orderType field; the distinction between SL and SL_M is not functionally meaningful in the current implementation.

7.12 User.totalPnl Tracking Inconsistency

The User.totalPnl field is updated inconsistently across different trade types. For Equity BUY: totalPnl is decremented by brokerage (treated as an immediate realized loss). For Equity SELL: totalPnl is incremented by the full realizedPnl (which already includes brokerage deduction). For Futures BUY/SELL and Options BUY: totalPnl is NOT updated at all at entry time (only totalTrades is incremented). For Options SELL (short): same as futures, no totalPnl update at entry. This means the totalPnl field does not accurately reflect the user's true P&L until all positions are closed, and during the period when futures/options positions are open, totalPnl understates the true cost by omitting brokerage. The portfolio endpoint compensates for this by calculating totalRealizedPnl from the aggregate of closed positions rather than relying on the User.totalPnl field.